

MBA EM **ENGENHARIA DE SOFTWARE**

Bancos de Dados Não-Relacionais II

Profa. Dra. Kelly Rosa Braghetto

A responsabilidade pela idoneidade, originalidade e licitude dos conteúdos didáticos apresentados é do professor.

Proibida a reprodução, total ou parcial, sem autorização.

[Lei nº 9610/98](#)

Introdução ao MongoDB

MongoDB

- Sistema gerenciador de banco de dados NoSQL orientado a documentos
- Esquema flexível / esquema opcional
- API de manipulação de dados em JavaScript
- Software livre (o MongoDB Community)
- MongoDB Enterprise – versão paga, com recursos adicionais voltados para empresas (ex: criptografia de disco e auditoria)
- Multiplataforma
- Criado em 2007
- Versão atual: 8.2
- Nome vem de “*huMONGOus*” – capaz de lidar com *enormes* volumes de dados
- <https://www.mongodb.com/>

Um BD no MongoDB

- Banco de dados = conjunto de coleções
- Coleção (*collection*) = conjunto de documentos
- Documento $\approx$ objeto em JSON
- Além das coleções:
 - Visões (somente leitura)
 - Visões materializadas (sob-demanda)

Tipos de dados do JSON

- **null**
- **bool** – *true* ou *false*
- **string** (codificação Unicode)
- **number** – decimal
 - pode ter sinal
 - pode ser escrito em notação científica (ex.: 2.5e-4)
- **object** – conjunto de itens do tipo chave-valor, onde cada chave é uma string única dentro do objeto
- **array** – lista ordenada de valores, possivelmente de tipos diferentes, delimitados por colchetes e separados por vírgulas

Exemplo de documento em JSON

```
{  
  "nome": "João", //string  
  "idade": 21, //number  
  "eleitor": true, //boolean  
  "escolaridade": null, //null  
  "hobbies": ["tênis", "xadrez"], //array  
  "endereço": {  
    "cidade": "São Paulo",  
    "estado": "SP"  
  } //object/document  
}
```

BSON – Binary JSON

- O BSON (Binary JSON) é o formato binário de *seriação* de documentos JSON
<http://bsonspec.org/>
- Seriação é
“o processo de tradução de estruturas de dados ou objetos em um formato que pode ser armazenado (por exemplo, em um arquivo ou buffer de memória) ou transmitido (por exemplo, via uma conexão de rede) e reconstruído posteriormente, possivelmente em um ambiente computacional diferente”
<https://en.wikipedia.org/wiki/Serialization>
- O BSON é usado no MongoDB para o armazenamento e transmissão de dados

BSON – Binary JSON

O BSON possui tipos de dados não existentes em JSON:

- **objectId** usado para identificação de documentos no MongoDB
 - **date** data
 - **binData** vetor de bytes
 - **int** inteiros de 32 bits
 - **long** inteiros de 64 bits
 - **regex** expressão regular
 - **maxKey**
 - **minKey**
 - **timestamp**
 - ...
- } tipos usados apenas internamente pelo MongoDB

<https://docs.mongodb.com/manual/reference/bson-types/>

Identificação de objetos

- Todo documento em uma coleção deve ter, obrigatoriamente, um campo chamado **_id** que funciona como chave primária para o documento
 - O campo **_id** pode ser de qualquer tipo de dado
- Se na criação de um documento o campo **_id** não for definido, o MongoDB atribuirá um valor do tipo **objectId** para o campo
- Um valor gerado do tipo **objectId** ocupa 12 bytes e é composto por:
 - 4 bytes para representar o timestamp da criação do ID;
 - 5 bytes para um valor aleatório gerado uma só vez por processo. Esse valor é único para a máquina e o processo;
 - 3 bytes para um contador incremental local do processo.

Principais componentes do MongoDB

- **mongosh** : cliente do MongoDB (Mongo Shell)
- **mongod** : servidor do MongoDB
 - **mongod --replSet "nomeReplica"**: servidor de réplica do MongoDB
 - **mongod --configsvr**: serviço de configuração do particionamento de dados (ele processa as consultas do cliente e determina a localização dos dados)
- **mongos**: serviço de roteamento de consultas do cliente para o serviço de configuração do cluster
- **Lista de drivers**: <https://www.mongodb.com/pt-br/docs/drivers/>
São bibliotecas para a conexão de aplicações com BDs do MongoDB

Replicação / Particionamento no MongoDB

Métodos utilizados para a obtenção de escalabilidade horizontal

- **Sharding** – particionamento de dados
 - Divide uma coleção de documentos em diferentes partes (*shards*) e essas partes são distribuídas em servidores distintos.
 - Divisão é feita por meio de uma chave de *shard* (ex. UserId, data, região).
- **ReplicaSet** – conjunto de réplicas
 - É um grupo de servidores (mongod) que hospedam/armazenam o mesmo conjunto de dados.
 - Implementa o modelo Primário-Secundário, com replicação assíncrona.
 - Por padrão, lê do nó primário. Mas permite leitura de nós secundários.

Conexão com o MongoDB

Clientes de conexão do MongoDB:

- **MongoDB Shell (mongosh)**
 - Interface linha de comando
 - Vem no pacote *core* do MongoDB, mas pode ser instalado individualmente:
 - <https://www.mongodb.com/try/download/shell>
- **MongoDB Compass**
 - Interface gráfica (GUI)
 - <https://www.mongodb.com/try/download/compass>

MongoDB Atlas

- O MongoDB Atlas é um serviço de BD em nuvem do MongoDB

<https://www.mongodb.com/cloud/atlas>

- Para criar uma conta no Atlas e ter um cluster simples gratuito com uma instalação do MongoDB, siga todos os passos do tutorial da aba “**IU do Atlas**” da seguinte página:

<https://www.mongodb.com/pt-br/docs/atlas/getting-started/>

- Atenção ao passo 4 do tutorial, que habilita o seu *cluster* no Atlas a receber conexões de softwares instalados no seu computador pessoal
- Para informações sobre como estabelecer uma conexão entre o mongosh e um *cluster* MongoDB Atlas, acesse:

<https://docs.atlas.mongodb.com/mongo-shell-connection/>

Sobre o restante desta aula

- Os exemplos e comandos que veremos a seguir introduzem o uso da ferramenta **mongosh** (MongoDB Shell)
- Para reproduzi-los, se conecte a uma instância de servidor MongoDB usando o cliente **mongosh** instalado em sua máquina
 - Você pode se conectar à instância que está no seu *cluster* no Atlas ou a um servidor que você tenha instalado no seu próprio computador

Estabelecendo uma conexão

- Forma geral do comando para conexão:

```
$ mongosh  
mongodb://[username:password@]host1[:port1][,...hostN[:portN]][/[defaultauthdb][?options]  
]
```

- Para se conectar a uma instância de servidor rodando localmente, na porta default (27017), execute no terminal:

```
$ mongosh      ou  $ mongosh "mongodb://localhost:27017"
```

- Exemplo de conexão com uma instância de servidor no MongoDB Atlas

```
$ mongosh "mongodb+srv://my_cluster.mongodb.net/my_db" --username my_user
```

<https://docs.mongodb.com/manual/reference/connection-string/>

O mongo Shell

- Usado para:
 - Consultas a dados
 - Modificações de dados
 - Operações administrativas sobre os bancos de dados
- A linguagem usada no mongo Shell é JavaScript
- A ferramenta pode ser usada em modo:
 - Interativo, para execução de um comando por vez
 - *Scripting*, em que se passa para o **mongosh**, na linha de comando, um arquivo com código em JavaScript para ser executado

Com o mongo, pode-se...

- Executar scripts js
- Declarar variáveis
- Manipular objetos JSON

```
for(i=0; i < 3; i++){  
    print ("hello, " + i);  
}
```

```
var test = "abc";  
print (test);  
test
```

```
dict = {"a":1, "b":2};  
dict  
dict.a  
dict["a"]  
w = "a"  
dict[w]
```

Acessando o *help* em diferentes níveis

```
> //ajuda geral
> help;
> //ajuda nível do banco de dados
> db.help();
> //ajuda nível de uma coleção
> db.<nome_coleção>.help();
> //ajuda nível do comando find()
> db.<nome_coleção>.find().help();
> //consulta a definição de uma função
> db.<nome_coleção>.find;
```

<nome_coleção> é o nome da coleção dentro do banco de dados selecionado com
use <nome_bd>

Alguns comandos úteis

- Lista todos os BDs do servidor
> `show dbs`
- Conecta (ou cria, se ainda não existir) o BD especificado
> `use <nome_bd>`
- Obtém ponteiro para BD atualmente em uso
> `db`
- Lista todas as coleções de BD atualmente em uso
> `show collections`
- Sai do Shell
> `quit()`

Criação e uso de BDs

- Para usar um banco de dados já existente ou criar um novo banco de dados, basta fazer:

```
> use <nome_do_BD>;
```

Se ainda não existir um BD com o nome indicado, esse BD será criado assim que uma coleção for criada dentro dele.

Criação de coleções

- Pode-se incluir um documento em uma coleção no BD em uso
 - se a coleção não existe ainda, ela será automaticamente criada
- O comando abaixo cria a coleção `disciplinas` com um documento dentro dela:

```
> db.disciplinas.insertOne({ "nome": "Bancos de Dados" });
```

- Ou pode-se criar uma coleção vazia:

```
> db.createCollection("disciplinas");
```

Organizando as coleções com *namespaces*

- Pode-se agrupar as coleções em *namespaces*
 - A notação com ponto é usada para designar a qual *namespace* pertence uma coleção
- Exemplos (abaixo, “mba” é um *namespace*):

```
> db.mba.alunos.insertOne({"nusp":12345, "nome":"John Lennon"});  
> db.mba.disciplinas.insertOne({"nome":"Bancos de Dados"});
```

ou

```
> db.createCollection("mba.alunos");  
> db.createCollection("mba.disciplinas");
```

Remoção de coleções e BDs

- Para apagar a coleção `disciplinas`:

```
db.disciplinas.drop();
```

- Para apagar o banco de dados atualmente em uso:

```
db.dropDatabase();
```

Coleções de documentos para os exemplos da aula

- No material da aula, foi disponibilizado um contêiner docker com uma instalação do MongoDB
- Essa instalação contém um banco de dados chamado “mba_esalq”, que possui dentro dele três coleções de documentos que usaremos nos exemplos desta aula:
 - Coleção “bios”: contém dados da biografia de personalidades da Computação (incluindo seus prêmios e suas contribuições para a área)
 - Coleção “zipcodes”: com dados de CEPs dos EUA
 - Coleções “pessoas” e “compras”: com dados simples de pessoas, suas preferências de esporte e compras realizadas por elas

Exemplo de documento da coleção *bios*

```
{  "_id" : 6,
   "name" : {
     "first" : "Guido",
     "last" : "van Rossum" },
   "birth" : ISODate("1956-01-31T05:00:00Z"),
   "contribs" : [ "Python" ],
   "awards" : [
     {
       "award" : "Award for the Advancement of Free Software",
       "year" : 2001,
       "by" : "Free Software Foundation"
     },
     {
       "award" : "NLUUG Award",
       "year" : 2003,
       "by" : "NLUUG"
     }
   ]
}
```

Para os próximos exemplos

- Se conecte no mongo Shell ao banco de dados bd_bios criado com o script "criacao_colecao_bios.js"
 - Depois de rodar o script, entre no mongo Shell e execute:

> use bd_bios;

Busca de documentos

No MongoDB, toda busca se dá no escopo de uma única coleção.

- Para listar todos os documentos da coleção `bios`:

```
> db.bios.find({});
```

- Para listar as personalidades da coleção que possuem primeiro nome “James” e sobrenome “Gosling”:

```
> db.bios.find({"name":{"first": "James", "last":"Gosling"}});
```

ou

```
> db.bios.find({"name.first": "James", "name.last":"Gosling"});
```

Formato da resposta

- Para exibir a resposta em um formato mais legível (com um par chave:valor por linha), use função `pretty()` depois do `find()`:

```
> db.bios.find(  
  {"name.first": "James", "name.last": "Gosling"}  
)pretty();
```

Busca de documentos

- O comando abaixo procura os documentos na coleção `bios` que contêm um campo chamado `first` dentro de um objeto aninhado chamado `name` e que possuem valor `"James"` para esse campo:

```
> db.bios.find({"name.first": "James"});
```

- Este outro comando procura os documentos na coleção `bios` que contêm um objeto aninhado chamado `name` e que possui um valor exatamente igual a `{"first": "James"}`. Se um documento contiver outros campos além de `first` dentro de `name`, ele não aparecerá na resposta, ainda que `"first"` tenha o valor `"James"`

```
> db.bios.find({"name":{"first": "James"}});
```

Busca de documentos

- Formato mais geral da busca:

```
> db.bios.find(  
    {"birth": {$gte: ISODate("1930-01-01T00:00:00Z") }},  
    {"name":1, "contribs":1 }  
) .limit(5);
```

**Coleção
consultada**

**Condição de
seleção dos
documentos**

**Modificador
da resposta**

**Campos mostrados
na resposta
(projeção)**

<https://docs.mongodb.com/manual/tutorial/query-documents/>

Busca de documentos

- Busca com condição **OU** – Exemplo

Seleciona os nomes e as contribuições das personalidades que ganharam um prêmio Turing ou que morreram antes dos anos 2000.

```
> db.bios.find(  
  { $or: [{"awards.award": "Turing Award"},  
          {"death": { $lt: ISODate("2000-01-01T00:00:00Z") }}  
    ],  
  {"name": 1, "contribs": 1}  
) ;
```

Obs.: `$lt` é a operação *less than* (`<`)

Busca de documentos

- Seleciona os nomes e as contribuições das personalidades que ganharam um prêmio Turing ou que morreram antes dos anos 2000.

```
> db.bios.find(  
  { $or: [{"awards.award": "Turing Award"},  
          {"death": { $lt: ISODate("2000-01-01T00:00:00Z") }}  
    ],  
  {"name": 1, "contribs": 1}  
);
```

- Campos com valor 1 aqui são exibidos na resposta, enquanto os com valor 0 não são
- Campos que não aparecem aí não são exibidos na resposta pois zero é o valor default
- Exceção: o default para exibição do campo `_id` é 1
 - Para não mostrá-lo na resposta da consulta, é preciso incluir aí `"_id": 0`

Busca de documentos

- Busca com condição **E** e **OU** - Exemplo

Seleciona os nomes e as contribuições das personalidades que nasceram depois dos anos 30 e que (ganharam um prêmio Turing ou que morreram antes dos anos 2000).

```
> db.bios.find(  
  {  
    "birth": { $gte: ISODate("1930-01-01T00:00:00Z") },  
    $or: [ {"awards.award": "Turing Award"},  
           {"death": { $lt: ISODate("2000-01-01T00:00:00Z") }}  
  ],  
  {"name": 1, "contribs": 1}  
) ;
```

Obs.: \$gte é a operação greater than or equal to ($\geq$)

Busca de documentos

- Busca com condições **E** - Exemplo

Seleciona os nomes e as contribuições das personalidades que nasceram depois dos anos 30 e antes dos anos 80 e que ganharam um prêmio Turing.

```
> db.bios.find(  
{  "birth": {$gte: ISODate("1930-01-01T00:00:00Z"),  
          $lt: ISODate("1980-01-01T00:00:00Z") },  
  "awards.award": "Turing Award"},  
{"name":1, "contribs":1}  
);
```

Busca de documentos

- A consulta abaixo **não** gera o mesmo resultado que a do slide anterior porque quando aparece mais de uma condição para um mesmo atributo, somente a última delas vai ser considerada (no caso, `birth < 1980-01-01`):

```
> db.bios.find(  
  { "birth": {$gte: ISODate("1930-01-01T00:00:00Z")},  
    "birth": {$lt: ISODate("1980-01-01T00:00:00Z")},  
    "awards.award": "Turing Award" },  
  { "name": 1, "contribs": 1 }  
);
```

Operadores lógicos para condições de busca

Operadores lógicos são usados para unir cláusulas na condição de busca

- **\$or**
- **\$and**
- **\$not**
- **\$nor** – devolve os documentos para os quais todas as cláusulas unidas pelo **\$nor** são avaliadas como falsas

<https://docs.mongodb.com/manual/reference/operator/query/#query-selectors>

Operadores para condições de busca

- **\$gt** – *greater than* (>)
- **\$gte** – *greater than or equal* (>=)
- **\$lt** – *less than* (<)
- **\$lte** – *less than or equal* (<=)
- **\$eq** – *equal* (=)
- **\$ne** – *not equal* (<>)
- **\$in** – verifica a pertinência num conjunto de valores
- **\$nin** – verifica a não pertinência num conjunto de valores

<https://docs.mongodb.com/manual/reference/operator/query/#query-selectors>

Operadores para condições de busca – Exemplo: \$in

- Encontre as personalidades que possuem primeiro nome igual a "John" ou "Grace":

```
> db.bios.find(  
    {"name.first": {$in: [ "John", "Grace" ] } }  
);
```

Operadores para condições de busca

- **\$exists** – verifica a existência de atributos
- **\$elemMatch** – verifica se existe pelo menos um elemento no vetor que satisfaz a(s) condição(ões) de comparação
- **\$size** – compara tamanho de vetor
- **\$regex** – “casa” com expressão regular
- ...

<https://docs.mongodb.com/manual/reference/operator/query/#query-selectors>

Operadores para condições de busca – Exemplo: \$size

- Encontre os nomes e contribuições das personalidades que tiveram 3 contribuições para a Computação:

```
> db.bios.find(  
    {"contribs": {$size:3}},  
    {"name":1, "contribs":1, "_id":0}  
) ;
```

Operadores para condições de busca – Exemplo: \$elemMatch

- Encontre os nomes e contribuições das personalidades que criaram a linguagem de programação *Simula*:

```
> db.bios.find(  
  { "contribs": { $elemMatch: { $eq: "Simula" } } },  
  { "name": 1, "contribs": 1, "_id": 0 }  
);
```

Busca em uma posição específica de um array

- A consulta abaixo seleciona todas as personalidades cujo primeiro prêmio recebido foi um Turing Award. Ou seja, seleciona os documentos que contêm dentro da primeira posição do vetor `awards` (que é a posição 0) um objeto que contenha `award="Turing Award"`.

```
> db.bios.find(  
  {"awards.0.award": "Turing Award"},  
  {"name":1,"awards":1});
```

Busca por campos nulos ou inexistentes

- A consulta abaixo seleciona o campo `name` dos documentos que não possuem o campo `birth` ou que possuem o valor `null` para ele:

```
> db.bios.find({"birth":null},"name":1);
```

- A consulta abaixo seleciona o campo `name` dos documentos que possuem o valor `null` no campo `birth`, ou seja, que o valor do item é do tipo BSON Null (tipo número 10):

```
> db.bios.find({"birth":{"$type":10}},"name":1);
```

- A consulta abaixo seleciona o campo `name` dos documentos que não possuem o campo `birth`:

```
> db.bios.find({"birth":{"$exists":false}},"name":1);
```

Operadores para condições de busca – Exemplo: \$regex

- Encontre os nomes de personalidades que contêm "joh" no primeiro nome:

```
> db.bios.find(  
    {"name.first": {$regex: 'joh', $options: 'i'}},  
    {"_id": 0, "name": 1});
```

ou

```
> db.bios.find({"name.first": /joh/i},  
    {"_id": 0, "name": 1});
```

Obs.: a opção 'i' é para ignorar a diferença entre maiúsculas e minúsculas.

Operadores para condições de busca – Exemplo: \$regex

- Encontre os nomes de personalidades cujo primeiro nome termine com "n", mas que não comece com "J":

```
> db.bios.find(  
  { "name.first": { $regex: 'n$',  
    $not: { $regex: '^J' } } },  
  { "_id": 0, "name": 1 }  
);
```

Mais sobre o uso de regex de JavaScript pode ser visto em:

- <https://docs.mongodb.com/manual/reference/operator/query/regex/>
- http://www.w3schools.com/jsref/jsref_obj_regexp.asp

Modificadores de resultados de consulta - Exemplos

- Mostra o nome e data de nascimento da sexta personalidade (uma depois de 5) da lista de personalidades ordenadas descendentemente por data de nascimento.

```
> db.bios.find({}, {"name":1, "birth":1}) .  
    sort({"birth":-1}) .limit(1) .skip(5) ;
```

Obs: Somente são considerados nessa consulta os documentos em `bios` que possuem o campo `"birth"` (nem todos têm!)

Aggregation Framework

- O **Aggregation Framework** do MongoDB permite definir *pipelines* de processamento dos dados
 - *Pipelines* são compostos por etapas (*stages*) de processamento
 - Cada etapa transforma os dados à medida que eles passam por ela
- Pode ser usado para:
 - Consultar documentos
 - Modificar a estrutura dos documentos da resposta de uma consulta
 - Gerar resultados agregados
 - Modificar documentos no BD
 - Criar novos documentos no BD

<https://docs.mongodb.com/manual/aggregation/#aggregation-framework>

Aggregation Framework

- Forma geral do *pipeline*:

```
db.collection.aggregate( [ { <etapa1> }, { <etapa2> }, ... ] )
```

- Exemplos de tipos de etapas que podem ser usadas com a função **aggregate**:
 - **\$match**: passa para a próxima etapa do pipeline os documentos no stream que casam com o filtro
 - **\$group**: agrupa documentos e aplica operações de agregação sobre os grupos
 - **\$project**: remodela cada documento de entrada (adicionando novos campos ou removendo campos existentes). Gera um documento de saída para cada um de entrada
 - **\$sort**: reordena os documentos do stream pela chave escolhida
 - **\$limit**: passa os n primeiros documentos de entrada para o pipeline (onde n é o limite especificado).

<https://docs.mongodb.com/manual/meta/aggregation-quick-reference/#stages>

Aggregation Framework

- Forma geral do *pipeline*:

```
db.collection.aggregate( [ { <etapa1> }, { <etapa2> }, ... ] )
```

- Mais tipos de etapas que podem ser usadas com a função **aggregate**:
 - **\$unwind**: desconstrói um campo de *array* dos documentos de entrada para gerar como saída um documento para cada elemento no *array*
 - **\$geoNear**: devolve um stream ordenado de documentos com base na proximidade a um dado ponto geoespacial
 - **\$lookup**: Faz uma junção externa à esquerda com uma outra coleção (não particionada) do banco de dados
 - **\$out**: escreve os documentos resultantes do *pipeline* em uma coleção
 - **\$merge**: também escreve os documentos resultantes do *pipeline* em uma coleção, mas permite inserir novos documentos, intercalar documentos, substituir documentos já existentes, etc.

Agrupamentos & Agregações

- Uma etapa do tipo \$group permite fazer agrupamentos e agregações
- Exemplos:
 - soma de todos os valores dos itens de um pedido
 - média das notas por aluno de uma disciplina
- Essas operações se assemelham às de GROUP BY, HAVING, COUNT, AVG, SUM, MAX e MIN da linguagem SQL

<https://docs.mongodb.com/manual/aggregation/#aggregation-framework>

Agregação simples

- Forma geral de agregações simples (sem agrupamento):

```
db.collection.aggregate(  
  { $group :  
    { _id : null,  
      <nome_resultado_agregação1>:{  
        <função_agregação1>:"$<atributo_a_agregar>"},  
      <nome_resultado_agregação2>:{  
        <função_agregação2>:"$<atributo_a_agregar>"},  
      ...  
    }  
  }  
) ;
```

Banco de dados para os exemplos

- Para os exemplos de agregação/agrupamento, usaremos a coleção zipcodes disponível na documentação do MongoDB. Essa coleção contém informações sobre CEPs dos EUA.
- Estrutura de um documento da coleção zipcodes:

```
{  
  "_id": "10280",  
  "city": "NEW YORK",  
  "state": "NY",  
  "pop": 5574,  
  "loc": [  
    -74.016323,  
    40.710537  
  ]  
}
```

<https://docs.mongodb.com/manual/tutorial/aggregation-zip-code-data-set/>

Exemplo de agregação simples

- Calcula a soma do atributo `pop` (ou seja, calcula a população total) e nomeia o resultado de `totalPop`:

```
db.zipcodes.aggregate(  
  [  
    { $group: { _id: null,  
                totalPop: { $sum: "$pop" }  
            }  
  ]  
);
```

Resposta da consulta:

```
{ "_id" : null, "totalPop" : 248408400 }
```

Agregação com agrupamento

- Forma geral de agregações com agrupamento:

```
db.collection.aggregate(  
  { $group :  
    { _id : <atributo(s)_agrupador(es)>,  
      <nome_resultado_agregação1>:{  
        <função_agregação1>:"$<atributo_a_agregar>"},  
      <nome_resultado_agregação2>:{  
        <função_agregação2>:"$<atributo_a_agregar>"},  
      ...  
    }  
  }  
) ;
```

Exemplo de agrupamento

- Para cada estado, calcula a população total e a população média:

```
db.zipcodes.aggregate(  
  [  
    { $group: { _id: "$state",  
                stateTotalPop: { $sum: "$pop" },  
                stateAvgPop:    { $avg: "$pop" }  
            }  
    }  
  ]  
);
```

Exemplo de agrupamento

- Resposta parcial da consulta do slide anterior:

```
{ "_id" : "DE", "stateTotalPop" : 666168, "stateAvgPop" : 12569.207547169812 }
{ "_id" : "ID", "stateTotalPop" : 1006749, "stateAvgPop" : 4126.020491803279 }
{ "_id" : "MA", "stateTotalPop" : 6016425, "stateAvgPop" : 12692.879746835442 }
{ "_id" : "TN", "stateTotalPop" : 4876457, "stateAvgPop" : 8378.792096219931 }
{ "_id" : "TX", "stateTotalPop" : 16984601, "stateAvgPop" : 10164.333333333334 }
{ "_id" : "MS", "stateTotalPop" : 2573216, "stateAvgPop" : 7088.749311294766 }
{ "_id" : "WV", "stateTotalPop" : 1793146, "stateAvgPop" : 2733.454268292683 }
{ "_id" : "VA", "stateTotalPop" : 6181479, "stateAvgPop" : 7575.341911764706 }
{ "_id" : "IA", "stateTotalPop" : 2776420, "stateAvgPop" : 3011.301518438178 }
{ "_id" : "OH", "stateTotalPop" : 10846517, "stateAvgPop" : 10771.119165839125 }
...
```

Exemplo de filtro e depois agrupamento

- Para cada cidade do estado de New York (NY), calcule a população total:

```
db.zipcodes.aggregate(  
  [  
    { $match: { state: "NY" } },  
    { $group: { _id: "$city",  
                cityTotalPop: { $sum: "$pop" }  
            }  
    }  
  ]  
);
```

Exemplo de filtro e depois agrupamento

- Resposta parcial da consulta do slide anterior:

```
{ "_id" : "MONTROSE", "cityTotalPop" : 4350 }  
{ "_id" : "CONGERS", "cityTotalPop" : 8088 }  
{ "_id" : "KIRKWOOD", "cityTotalPop" : 4133 }  
{ "_id" : "MARILLA", "cityTotalPop" : 1063 }  
{ "_id" : "NAPLES", "cityTotalPop" : 4616 }  
{ "_id" : "PINE VALLEY", "cityTotalPop" : 252 }  
{ "_id" : "RAQUETTE LAKE", "cityTotalPop" : 0 }  
{ "_id" : "BINGHAMTON", "cityTotalPop" : 83017 }  
{ "_id" : "SCHUYLER FALLS", "cityTotalPop" : 652 }  
{ "_id" : "GENOA", "cityTotalPop" : 1160 }  
{ "_id" : "BELLPORT", "cityTotalPop" : 7927 }  
{ "_id" : "SENECA FALLS", "cityTotalPop" : 10987 }
```

...

Exemplo de filtro de grupo

- Para cada estado, calcula a população total e a população média, mas filtra o resultado (com o operador **\$match**), mostrando apenas os dados dos estados que possuem população total superior a 10 milhões de habitantes:

```
db.zipcodes.aggregate(  
  [  
    { $group: { _id: "$state",  
                stateTotalPop: { $sum: "$pop" },  
                stateAvgPop: { $avg: "$pop" }  
            }  
    },  
    { $match: { stateTotalPop: { $gt: 10000000 } } }  
  ]  
);
```

Exemplo de filtro de grupo

- Resposta da consulta do slide anterior:

```
{ "_id" : "TX", "stateTotalPop" : 16984601, "stateAvgPop" : 10164.333333333334 }
```

```
{ "_id" : "IL", "stateTotalPop" : 11427576, "stateAvgPop" : 9238.137429264349 }
```

```
{ "_id" : "PA", "stateTotalPop" : 11881643, "stateAvgPop" : 8149.275034293552 }
```

```
{ "_id" : "FL", "stateTotalPop" : 12686644, "stateAvgPop" : 15779.407960199005 }
```

```
{ "_id" : "OH", "stateTotalPop" : 10846517, "stateAvgPop" : 10771.119165839125 }
```

```
{ "_id" : "NY", "stateTotalPop" : 17990402, "stateAvgPop" : 11279.248902821317 }
```

```
{ "_id" : "CA", "stateTotalPop" : 29754890, "stateAvgPop" : 19627.236147757256 }
```

Exemplo de agrupamento envolvendo mais de um campo

- Calcule a população total de cada cidade.

Problemas: cada cidade pode conter mais de um zip-code e podem existir duas cidades com o mesmo nome em estados diferentes.

- **Solução: agrupar por estado e cidade!**

```
db.zipcodes.aggregate(  
    [  
        { $group: { _id: { gState: "$state", gCity: "$city" },  
                      cityPop: { $sum: "$pop" } }  
    }  
    ]  
);
```

Exemplo de agrupamento envolvendo mais de um campo

- Resposta parcial da consulta do slide anterior:

```
{ "_id" : { "gState" : "WA", "gCity" : "CATHLAMET" }, "cityPop" : 2114 }
```

```
{ "_id" : { "gState" : "IL", "gCity" : "WOODSTOCK" }, "cityPop" : 21770 }
```

```
{ "_id" : { "gState" : "NY", "gCity" : "DANSVILLE" }, "cityPop" : 8804 }
```

```
{ "_id" : { "gState" : "AL", "gCity" : "OAKMAN" }, "cityPop" : 3700 }
```

```
{ "_id" : { "gState" : "OH", "gCity" : "MARTINS FERRY" }, "cityPop" : 10605 }
```

```
{ "_id" : { "gState" : "SD", "gCity" : "EDGEMONT" }, "cityPop" : 1264 }
```

```
{ "_id" : { "gState" : "LA", "gCity" : "SINGER" }, "cityPop" : 2374 }
```

```
{ "_id" : { "gState" : "NH", "gCity" : "GILMANTON" }, "cityPop" : 1308 }
```

...

Exemplo de *pipeline* com duas etapas de agrupamento & agregação

- Para cada estado, calcule a população média das cidades do estado.

Problema: cada cidade pode ter mais de um zip-code.

- **Solução: pipeline de agregações/agrupamentos!**

```
db.zipcodes.aggregate(  
  [  
    { $group: { _id: { gState: "$state", gCity: "$city" },  
                cityPop: { $sum: "$pop" } }  
  },  
    { $group: { _id: "$_id.gState",  
                avgCityPop: { $avg: "$cityPop" } }  
  }  
  ]  
);
```

Exemplo de *pipeline* com duas etapas de agrupamento & agregação

```
db.zipcodes.aggregate (
```

```
[
```

```
  { $group: { _id: { gState: "$state", gCity: "$city" },  
                cityPop: { $sum: "$pop" } }  
  },
```

```
  { $group: { _id: "$_id.gState",  
                avgCityPop: { $avg: "$cityPop" } }  
  }
```

```
]
```

```
) ;
```

Esse bloco agrupa os zipcodes por estado e cidade e calcula a população total de cada grupo.

Resultado do bloco: conjunto de triplas (gState,gCity,cityPop).

Esse bloco agrupa o resultado do bloco anterior por estado e calcula a média da população em cada grupo.

Resultado do bloco: conjunto de duplas (gState,avgCityPop).

Exemplo de *pipeline* com duas etapas de agrupamento & agregação

- Resposta parcial da consulta do slide anterior:

```
{ "_id" : "IA", "avgCityPop" : 3123.0821147356583 }
{ "_id" : "SD", "avgCityPop" : 1839.6746031746031 }
{ "_id" : "VA", "avgCityPop" : 8526.177931034483 }
{ "_id" : "WV", "avgCityPop" : 2771.4775888717154 }
{ "_id" : "OR", "avgCityPop" : 8262.561046511628 }
{ "_id" : "MS", "avgCityPop" : 7524.023391812865 }
{ "_id" : "TX", "avgCityPop" : 13775.02108678021 }
{ "_id" : "TN", "avgCityPop" : 9656.350495049504 }
{ "_id" : "MA", "avgCityPop" : 14855.37037037037 }
{ "_id" : "ID", "avgCityPop" : 4320.811158798283 }
...
```

Funções de agregação

- Exemplos de funções (de agregação e acumuladoras) que podem ser usadas na etapa \$group:
 - Iguais às da SQL: **\$sum**, **\$avg**, **\$min**, **\$max**
 - **\$push**: devolve um vetor com todos os valores que apareceram no grupo para o campo escolhido
 - **\$addToSet**: devolve um vetor com todos os valores que apareceram no grupo para o campo escolhido, mas sem repetições de valores
 - **\$first**: retorna o valor de um campo do primeiro documento de um conjunto ordenado de documentos
 - **\$last**: retorna o valor de um campo do último documento de um conjunto ordenado de documentos

<https://docs.mongodb.com/manual/reference/operator/aggregation/#aggregation-pipeline-operator-reference>

Exemplo de uso das funções \$push e \$addToSet

- Para cada estado, obtém um vetor com todas as cidades do estado com repetições e outro vetor com as cidades sem repetições:

```
db.zipcodes.aggregate(  
  [  
    { $group: { _id: "$state",  
                bagOfCities: { $push: "$city" },  
                setOfCities: { $addToSet: "$city" }  
            }  
    }  
  ]  
);
```

Exemplo de uso das funções \$push e \$addToSet

- Um documento da resposta da consulta do slide anterior:

```
{  
  "_id" : "DC",  
  "bagOfCities" : [ "WASHINGTON", "WASHINGTON", "WASHINGTON", "WASHINGTON",  
"WASHINGTON", "WASHINGTON", "WASHINGTON", "WASHINGTON", "WASHINGTON", "WASHINGTON",  
"WASHINGTON", "WASHINGTON", "WASHINGTON", "WASHINGTON", "WASHINGTON", "WASHINGTON",  
"WASHINGTON", "WASHINGTON", "WASHINGTON", "WASHINGTON", "WASHINGTON", "WASHINGTON",  
"PENTAGON", "WASHINGTON" ],  
  "setOfCities" : [ "WASHINGTON", "PENTAGON" ]  
}
```

Exemplo de *pipeline* com várias etapas

```
db.zipcodes.aggregate( [  
  { $group:  
    { _id: { gState: "$state", gCity: "$city" },  
      cityPop: { $sum: "$pop" } } },  
  { $sort: { cityPop: 1 } },  
  { $group:  
    { _id : "$_id.gState",  
      biggestCity: { $last: "$_id.gCity" }, biggestPop: { $last: "$cityPop" },  
      smallestCity: { $first: "$_id.gCity" }, smallestPop: { $first: "$cityPop" } } },  
  { $project:  
    { _id: 0,  
      state: "$_id",  
      biggestCity: { name: "$biggestCity", pop: "$biggestPop" },  
      smallestCity: { name: "$smallestCity", pop: "$smallestPop" } } }  
] );
```

Para cada estado, obtém a cidade com a maior e a cidade com a menor população, junto com as suas respectivas populações.

Exemplo de *pipeline* com várias etapas

- Exemplo de documento que aparece na resposta da consulta do slide anterior:

```
{  
  "biggestCity" : {  
    "name" : "LAS VEGAS",  
    "pop" : 597557  
  },  
  "smallestCity" : {  
    "name" : "TUSCARORA",  
    "pop" : 1  
  },  
  "state" : "NV"  
}
```

Exemplo de *pipeline* envolvendo operações sobre *arrays*

- Mais dados para exemplos: pessoas e suas preferências de esportes

```
db.pessoas.insertMany([
  { "_id" : 1, "nome" : "jane", "esporte" : ["golf", "vôlei"] },
  { "_id" : 2, "nome" : "joe", "esporte" : ["tênis", "golf",
    "natação"] },
  { "_id" : 3, "nome" : "anne", "esporte" : ["vôlei", "basquete",
    "futebol", "golf"] },
  { "_id" : 4, "nome" : "peter", "esporte" : ["futebol", "vôlei"]}
]);
```

Exemplo de *pipeline* envolvendo operações sobre *arrays*

- Consulta: obtenha os dois esportes preferidos das pessoas cadastradas no BD (ou seja, os que têm mais pessoas que gostam dele)
 - **Solução: \$unwind para desconstruir o array de esportes e depois agrupar os seus valores !**

```
db.pessoas.aggregate(  
  [  
    { $unwind : "$esporte" },  
    { $group : { _id : "$esporte" ,  
                 frequencia : { $sum : 1 } } },  
    { $sort : { frequencia : -1 } },  
    { $limit : 2 }  
  ]  
);
```

Exemplo de *pipeline* envolvendo operações sobre *arrays*

```
db.pessoas.aggregate([  
  { $unwind : "$esporte" },  
  { $group :  
    { _id : "$esporte" ,  
      frequencia : { $sum : 1 } } },  
  { $sort : { frequencia : -1 } },  
  { $limit : 2 }  
]);
```



Resultado da operação \$unwind:

```
{ "_id" : 1, "nome" : "jane", "esporte" : "golf" }  
{ "_id" : 1, "nome" : "jane", "esporte" : "vôlei" }  
{ "_id" : 2, "nome" : "joe", "esporte" : "tênis" }  
{ "_id" : 2, "nome" : "joe", "esporte" : "golf" }  
{ "_id" : 2, "nome" : "joe", "esporte" : "natação" }  
{ "_id" : 3, "nome" : "anne", "esporte" : "vôlei" }  
{ "_id" : 3, "nome" : "anne", "esporte" : "basquete" }  
{ "_id" : 3, "nome" : "anne", "esporte" : "futebol" }  
{ "_id" : 3, "nome" : "anne", "esporte" : "golf" }  
{ "_id" : 4, "nome" : "peter", "esporte" : "futebol" }  
{ "_id" : 4, "nome" : "peter", "esporte" : "vôlei" }
```

Resultado final da consulta:

```
{ "_id" : "vôlei", "frequencia" : 3 }  
{ "_id" : "golf", "frequencia" : 3 }
```

Junção externa de coleções

- A etapa **\$lookup** faz uma junção externa à esquerda, com uma condição de igualdade entre o valor de um campo dos documentos da coleção de entrada e o valor de um campo dos documentos da coleção a ser “juntada”
- Cada documento resultante é um documento da entrada com um novo *array* inserido nele, contendo os documentos da outra coleção que “casaram” com ele na junção
- Formato geral:

```
{  
  $lookup:  
  {  
    from: <coleção para a junção>,  
    localField: <nome do campo para a junção na coleção de entrada>,  
    foreignField: <nome do campo para a junção na coleção do "from">,  
    as: <nome do campo de array a ser criado nos documentos resultantes>  
  }  
}
```

<https://docs.mongodb.com/manual/reference/operator/aggregation/lookup/>

Exemplo de *pipeline* envolvendo junção externa de coleções

- Mais dados para exemplos: compras feitas por pessoas

```
db.compras.insertMany([
  {"_id" : 1, "id_comprador": 2, "data": ISODate("2020-01-15"), "valor": "76.25"},
  {"_id" : 2, "id_comprador": 4, "data": ISODate("2020-01-07"), "valor": "10.05"},
  {"_id" : 3, "id_comprador": 2, "data": ISODate("2020-03-28"), "valor": "512.38"},
  {"_id" : 4, "id_comprador": 2, "data": ISODate("2020-04-21"), "valor": "62.99"},
  {"_id" : 5, "id_comprador": 4, "data": ISODate("2020-05-19"), "valor": "374.81"},
]);
```

Exemplo de *pipeline* envolvendo junção externa de coleções

- Consulta: Obtenha os dados das pessoas junto com os dados das compras que elas fizeram

```
db.pessoas.aggregate([
  {
    $lookup:
      {
        from: "compras",
        localField: "_id",
        foreignField: "id_comprador",
        as: "historico_compras"
      }
  }
]);
```

Parte da resposta da consulta do slide anterior:

```
{  "_id" : 3, "nome" : "anne",
  "esporte" : ["vôlei", "basquete", "futebol", "golf"],
  "historico_compras" : [ ]}
{  "_id" : 4, "nome" : "peter",
  "esporte" : ["futebol", "vôlei"],
  "historico_compras" : [
    {
      "_id" : 2, "id_comprador" : 4,
      "data" : ISODate("2020-01-07T00:00:00Z"),
      "valor" : "10.05"
    },
    {
      "_id" : 5, "id_comprador" : 4,
      "data" : ISODate("2020-05-19T00:00:00Z"),
      "valor" : "374.81"
    }
  ]
}
```

Exemplo de *pipeline* envolvendo junção externa de coleções

- Consulta: Obtenha os dados das pessoas junto com os dados das compras que elas fizeram.
 - **Versão 2: usa uma etapa de \$match para só incluir na resposta pessoas que já fizeram alguma compra**

```
db.pessoas.aggregate([
  { $lookup: {
    from: "compras",
    localField: "_id",
    foreignField: "id_comprador",
    as: "historico_compras" } },
  { $match: { "historico_compras": { $ne: [] } } }
]);
```

Exemplo de *pipeline* envolvendo junção externa de coleções

- Consulta: Obtenha os dados das pessoas junto com os dados das compras que elas fizeram a partir de abril de 2020.

```
db.pessoas.aggregate([
```

```
{ $lookup: {
```

```
  from: "compras",
```

```
  let: { id_pessoa: "$_id" },
```

```
  pipeline: [
```

```
    { $match: { $expr:
```

```
      { $and: [
```

```
        { $eq: [ "$id_comprador", "$$id_pessoa" ] },
```

```
        { $gte: [ "$data", ISODate("2020-04-01") ] } ] } } },
```

```
    { $project: { "id_comprador": 0, "_id": 0 } } ],
```

```
  as: "compras_recientes" } }],
```

```
{ $match: { "compras_recientes": { $ne: [] } } }
```

```
]);
```

Define variáveis para os campos da coleção **pessoas** que serão usados na condição de junção

Condição de junção/seleção de tuplas da coleção **compras**:

ID do comprador == ID da pessoa
Data da compra >= 01/04/2020

Define a estrutura dos objetos da coleção **compras** que serão incluídos na resposta da junção

Exemplo de *pipeline* envolvendo junção externa de coleções

- Resposta da consulta do slide anterior:

```
{
  "_id" : 2,
  "nome" : "joe",
  "esporte" : ["tênis", "golf", "natação"],
  "compras_recentes" : [{"data" : ISODate("2020-04-21T00:00:00Z"), "valor" : "62.99"}]
}

{
  "_id" : 4,
  "nome" : "peter",
  "esporte" : ["futebol", "vôlei"],
  "compras_recentes" : [{"data" : ISODate("2020-05-19T00:00:00Z"), "valor" : "374.81"}]
}
```



Referências Bibliográficas - MongoDB

- Documentação do MongoDB
 - <https://docs.mongodb.com/>
- Tutoriais oficiais:
 - <https://docs.mongodb.com/manual/>
 - <https://docs.mongodb.com/getting-started/shell/>
- Mapeamento de SQL para MongoDB
 - http://s3.amazonaws.com/info-mongodb-com/sql_to_mongo.pdf
 - <https://docs.mongodb.com/manual/reference/sql-comparison/>
- Livro:
 - “MongoDB: The Definitive Guide, 3rd Edition (2019) – Powerful and Scalable Data Storage“, de Kristina Chodorow, Editora: O'Reilly Media

Introdução ao Neo4J

Neo4j

- SGBD para grafos criado em 2007, mantido pela Neo Technology
- Software livre
- Usa como modelo grafos rotulados de propriedades
- Tem modelo de armazenamento e processamento nativos para grafos
- Usa uma linguagem de consulta chamada Cypher

A Linguagem Cypher

- Linguagem declarativa, como a SQL
 - Comandos definem o que deve ser recuperado, não como
- Muito legível e expressiva
- É usada para encontrar padrões nos relacionamentos de um grafo
- Tem comandos inspirados em outras linguagens de consulta conhecidas, como a SQL e a SPARQL
- Sua especificação é aberta
 - Foi desenvolvida como uma proposta de linguagem de consulta padrão para BDs de grafos

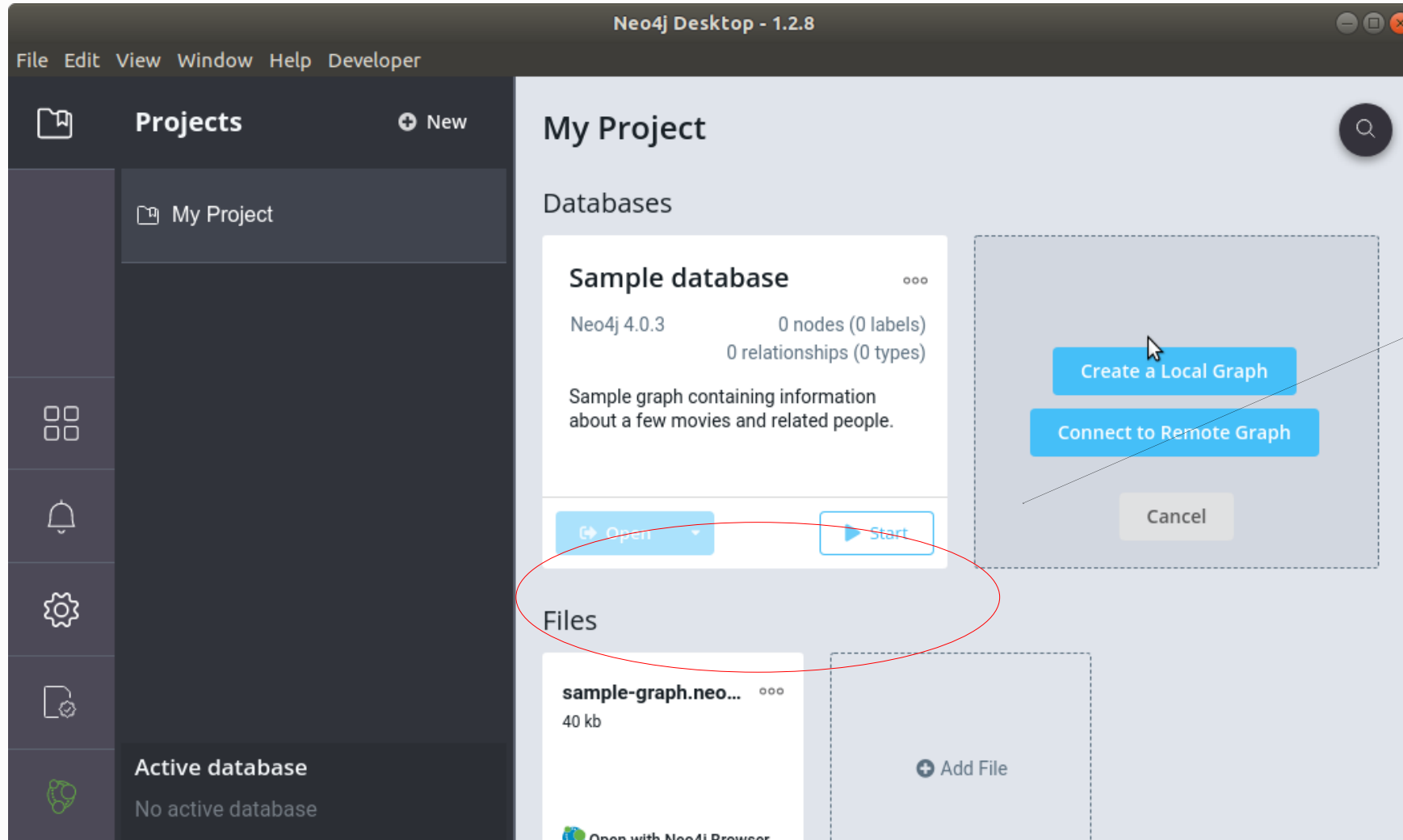
Instruções para Uso do Neo4j no seu computador

- Para executar os exemplos e exercícios sobre Neo4j, você pode instalar em seu computador o **Neo4j Desktop**
 - O Neo4j Desktop é uma ferramenta prática para se trabalhar com BDs locais
 - Ela **não** requer a instalação do Neo4j Server
- Alternativa: instale um servidor **Neo4j (Neo4j Community Edition)** em seu computador
- Para quem prefere clientes de conexão em modo texto: Cypher Shell

<https://neo4j.com/deployment-center/>

Neo4j Desktop

Abertura do BD para os Exemplos

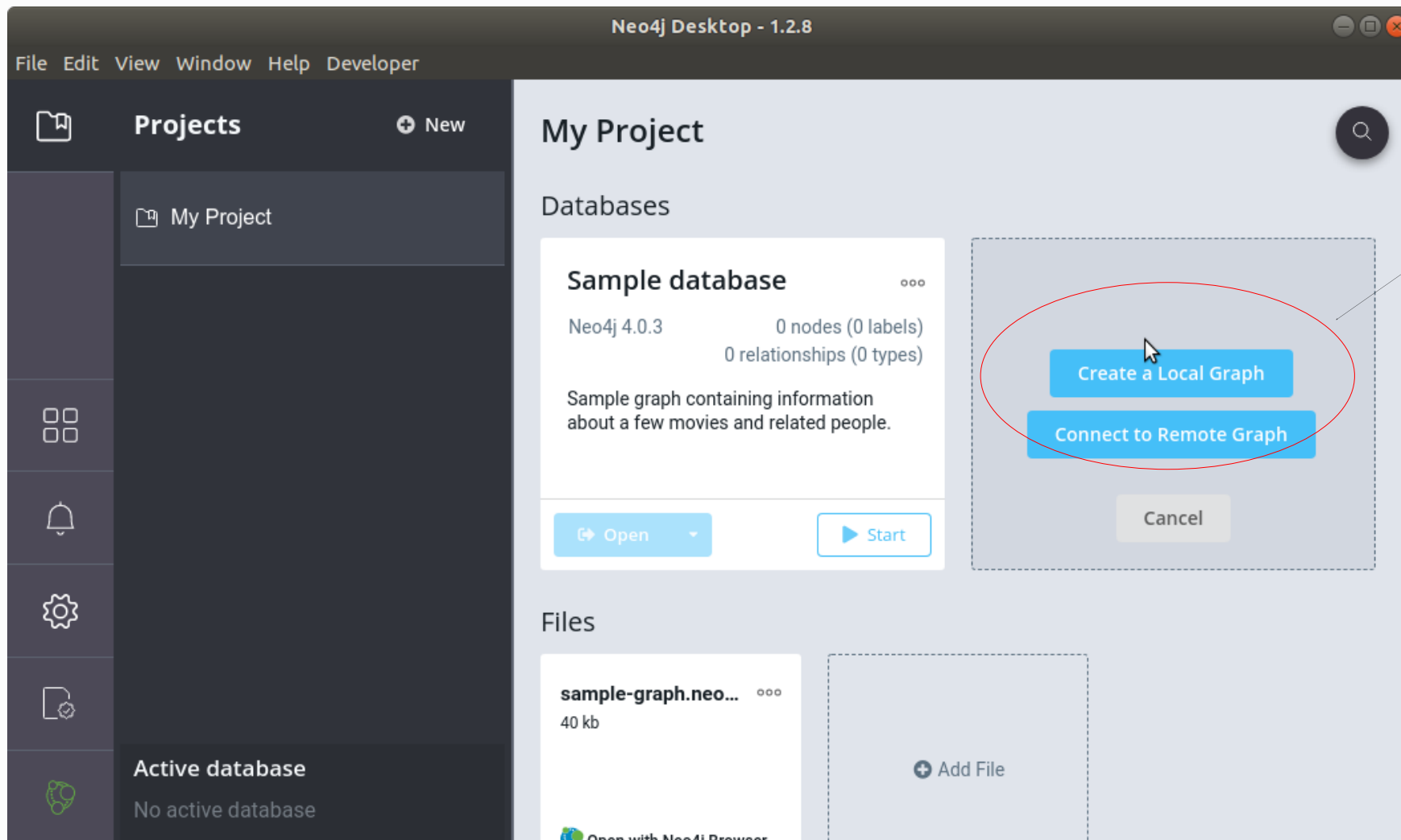


1) Clique em Start para criar o BD de exemplo (*Sample database*) que vem com a ferramenta

2) Depois, clique em Open para abrir a janela para o envio de operações e visualização de dados do BD

Neo4j Desktop

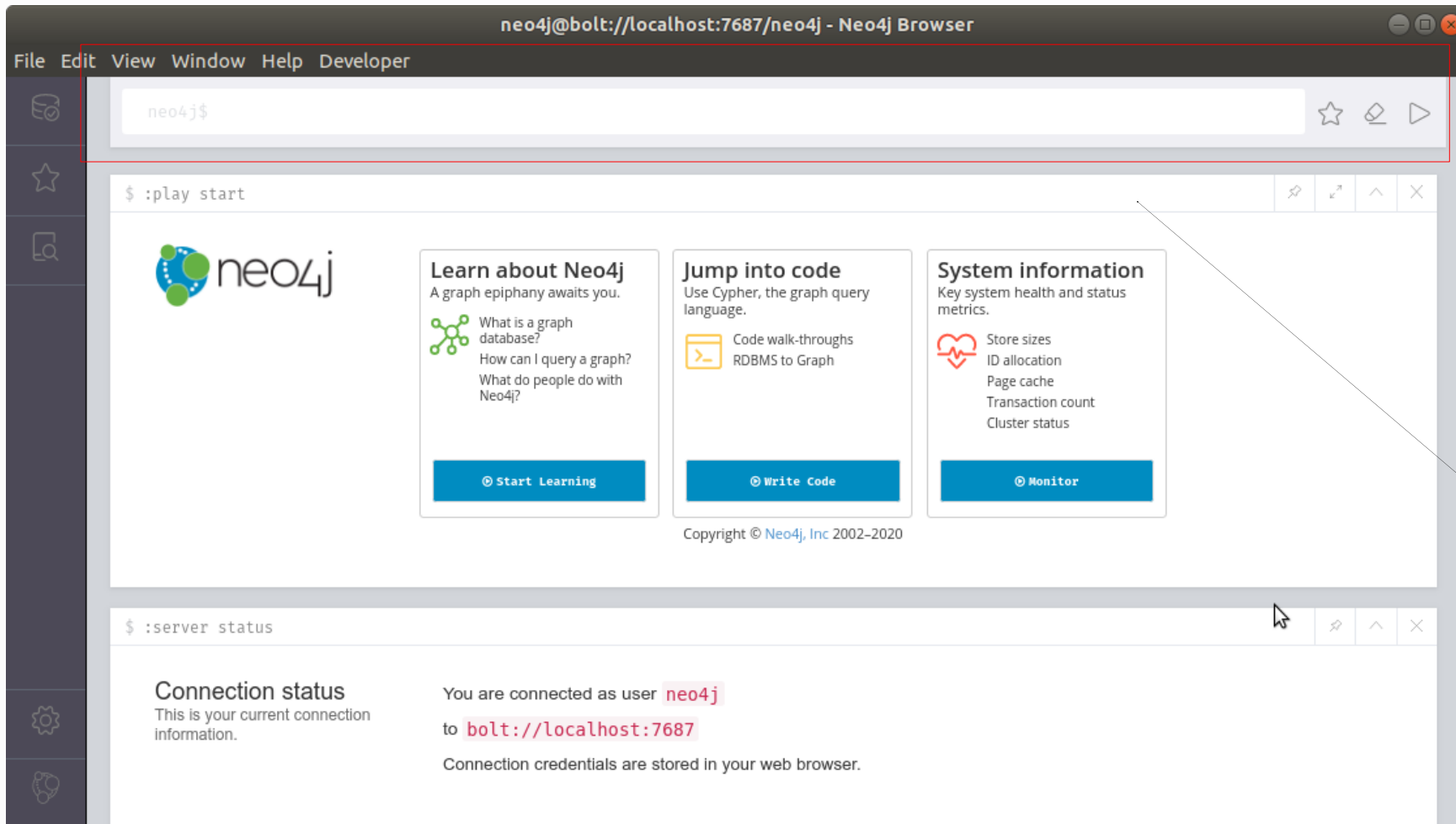
Alternativa (Passo 1): Crie o BD para os Exemplos



- 1) Crie um BD chamado *movies*
- 2) Como mostrado no slide anterior, faça o *Start* dele e, depois,
- 3) Clique em *Open* para abrir a janela para o envio de operações e visualização de dados do BD

Neo4j Desktop

Alternativa (Passo 2): Crie os Dados no Novo BD



Quem criou um novo BD (em vez de usar o *Sample database*), precisará povoá-lo com os dados do arquivo “grafo_filmes.txt”, disponibilizado com o material desta aula. Isso pode ser feito de duas formas. Na janela aberta para o BD:

- Abra o arquivo com os dados, copie o seu conteúdo e cole-o no campo de texto de texto destacado na imagem ao lado, OU
- Faça um *drag-and-drop* do arquivo sobre o campo.

Depois, aperte o botão *Play* para executar os comandos.

Instruções para uso do Neo4j no seu computador

1) Iniciar o SGBD executando o comando:

```
$ neo4j start
```

1) Abrir no browser o seguinte endereço: <http://localhost:7474/browser/>

1) Se logar no sistema usando *neo4j* como login e senha

- Criar o BD de filmes copiando e colando os comandos do script de criação “grafo_filmes.txt”

BD de Exemplo para a Aula

- Exemplos de comandos que estão no arquivo “grafo_filmes.txt”:

```
CREATE (TheMatrix:Movie {title:'The Matrix', released:1999,
tagline:'Welcome to the Real World'})
CREATE (Keanu:Person {name:'Keanu Reeves', born:1964})
CREATE (Carrie:Person {name:'Carrie-Anne Moss', born:1967})
CREATE (Laurence:Person {name:'Laurence Fishburne', born:1961})
CREATE (AndyW:Person {name:'Andy Wachowski', born:1967})
CREATE
  (Keanu)-[:ACTED_IN {roles:['Neo']}]>(TheMatrix),
  (Carrie)-[:ACTED_IN {roles:['Trinity']}]>(TheMatrix),
  (Laurence)-[:ACTED_IN {roles:['Morpheus']}]>(TheMatrix),
  (AndyW)-[:DIRECTED]>(TheMatrix)
```

Outra alternativa (fácil) para executar os exemplos desta aula

- Também é possível acessar o BD Movies no ambiente de demonstração do Neo4J,
 - Basta clicar no link abaixo

<https://demo.neo4jlabs.com:7473/browser/?dbms=neo4j://movies@demo.neo4jlabs.com&db=movies>

- Para o login, copiar o que aparece no campo “database user” e colar no campo “password”. Em “protocol”, selecione a opção “neo4j+[s://](#)” ou “https://”.
- Há outros BDs disponíveis no ambiente de demonstração
 - Mais detalhes em:

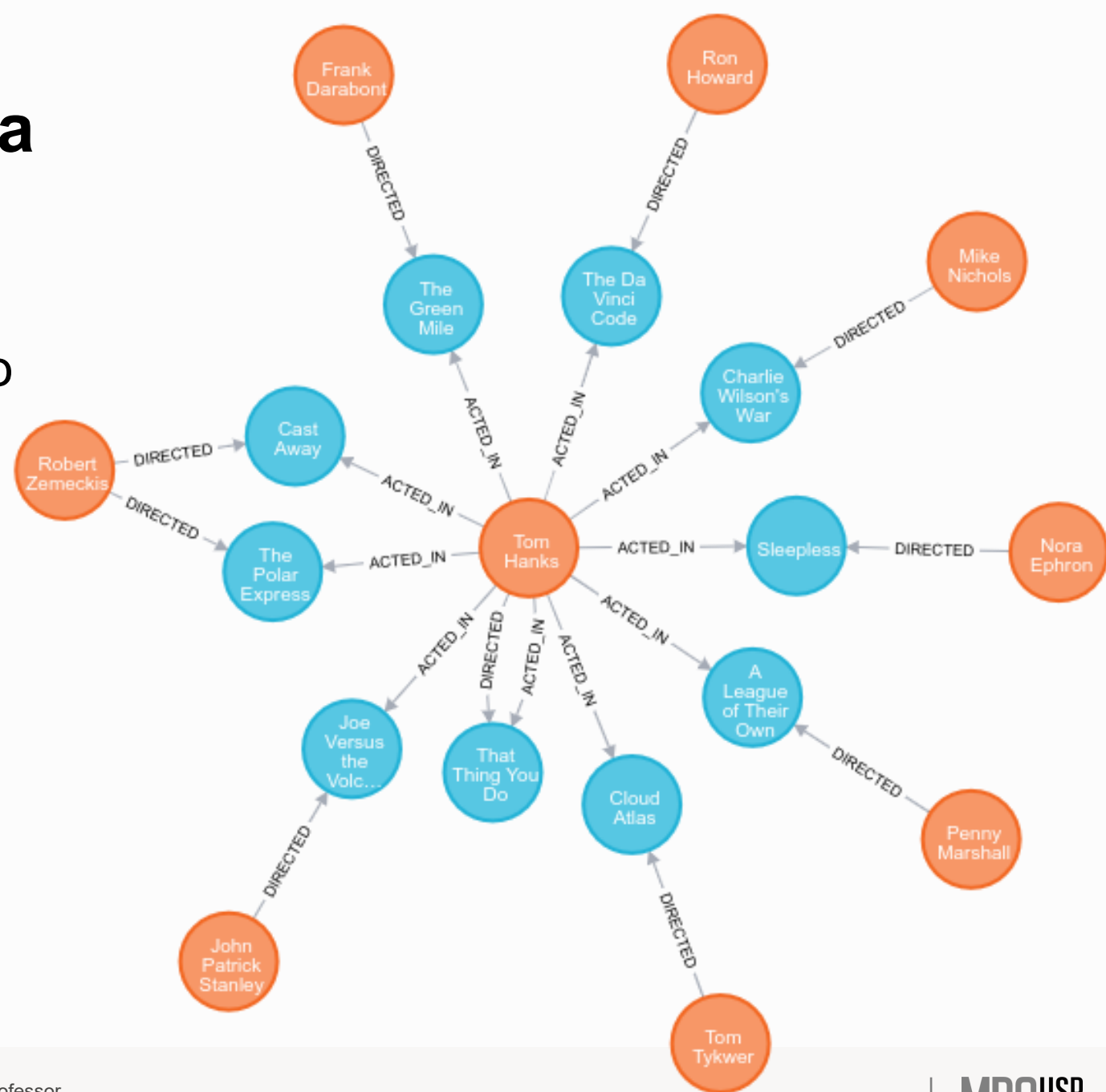
<https://neo4j.com/docs/getting-started/appendix/example-data/>

Mais uma Alternativa – em Nuvem

- Neo4J AuraDB - *Fully Managed Graph Database as-a-service*
 - <https://neo4j.com/product/auradb/>
 - Possível criar conta gratuita para teste

BD de Exemplo para a Aula

- A imagem mostra parte do BD criado
- O BD contém dados sobre filmes, atores e diretores
- Na interface gráfica do Neo4j, é possível interagir com os nós, consultar suas propriedades, etc.



Especificação de Nós em Cypher

- São denotados por um par de parênteses
- Exemplo: um nó com nome de variável `matrix`, rótulo `Movie` e duas propriedades (`title` e `released`)

```
(matrix:Movie {title: "The Matrix", released: 1999})
```

- A variável de nó pode ser usada em outros lugares dentro do mesmo comando em que foi definida
- O rótulo funciona como a definição de um tipo ou papel para o nó
 - Possibilita agrupar os nós
 - Bastante usado como filtro nas consultas
- Um nó pode ter mais de um rótulo
- Com exceção dos parênteses, todos os demais elementos de definição de nó mostrados acima são opcionais

Cypher

Outros Exemplos de Nós

`()`

`(matrix)`

`(:Movie)`

`(matrix:Movie)`

`(matrix:Movie {title: "The Matrix"})`

`(matrix:Movie {title: "The Matrix", released: 1999})`

`(matrix:Movie:Promoted)`

Cypher

Busca por Nós

- Consultas/buscas são realizadas por meio da função MATCH
- A consulta a seguir devolve todos os nós de filme (ou seja, os nós que têm `Movie` como rótulo)

```
MATCH (movie:Movie)
```

```
RETURN movie
```

Cypher

Busca por Nós

- A consulta a seguir devolve todos os nós de filme que têm a propriedade title = "The Matrix"

```
MATCH (movie:Movie {title:"The Matrix"})
```

```
RETURN movie
```

- O Cypher usa os rótulos na otimização das consultas, portanto o uso deles é fundamental para garantir um bom desempenho nas operações sobre o grafo
 - Na consulta acima, buscou-se somente os nós com rótulo `Movie`

Cypher

Busca por Relacionamentos

- Formato geral:

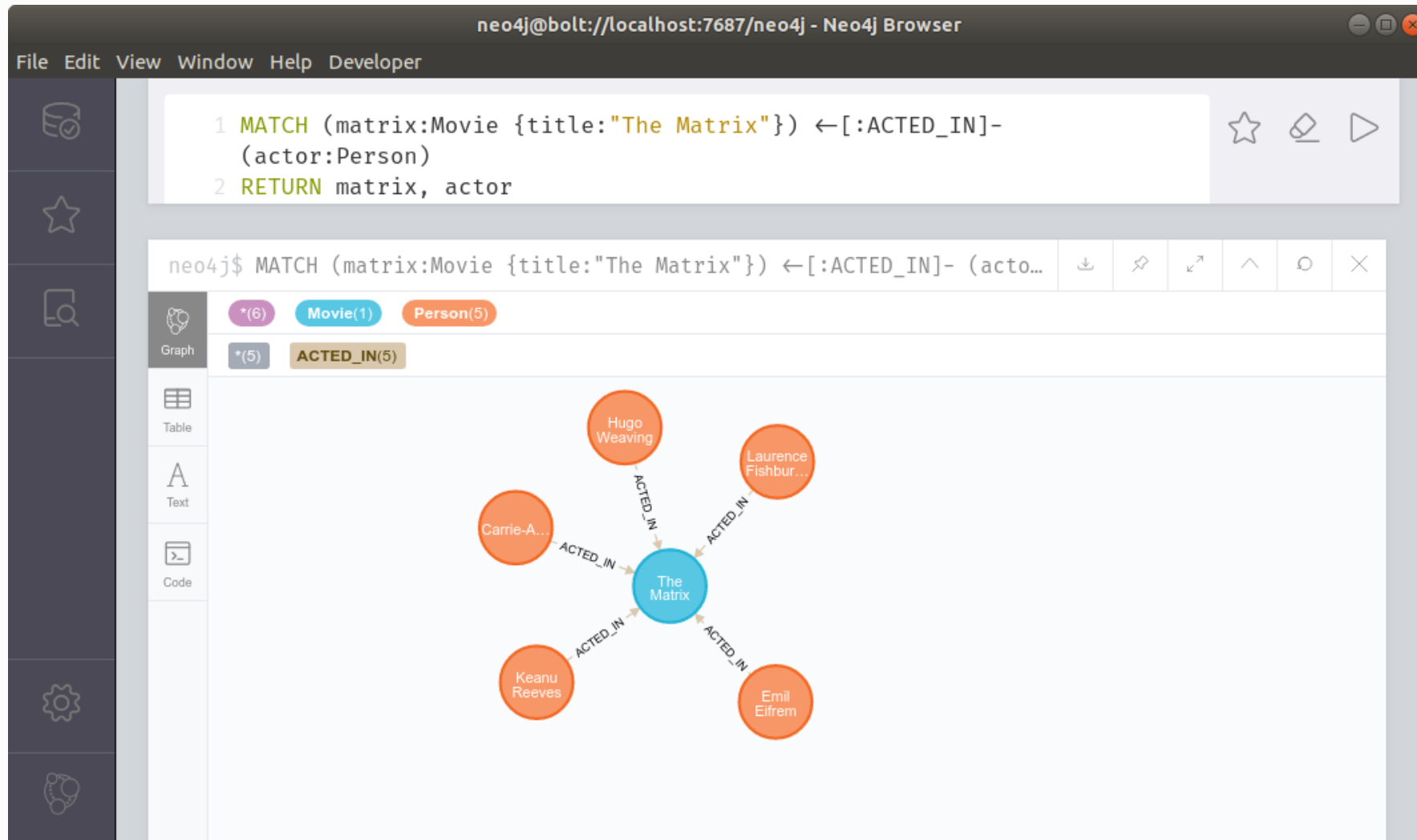
```
MATCH (node1:Label1) -[rel:REL_TYPE]-> (node2:Label2)
RETURN node1, node2
```

- Exemplo: Devolva todos os atores do filme *The Matrix*

```
MATCH (matrix:Movie {title:"The Matrix"}) <-[:ACTED_IN]-
(actor:Person)
RETURN matrix, actor
```

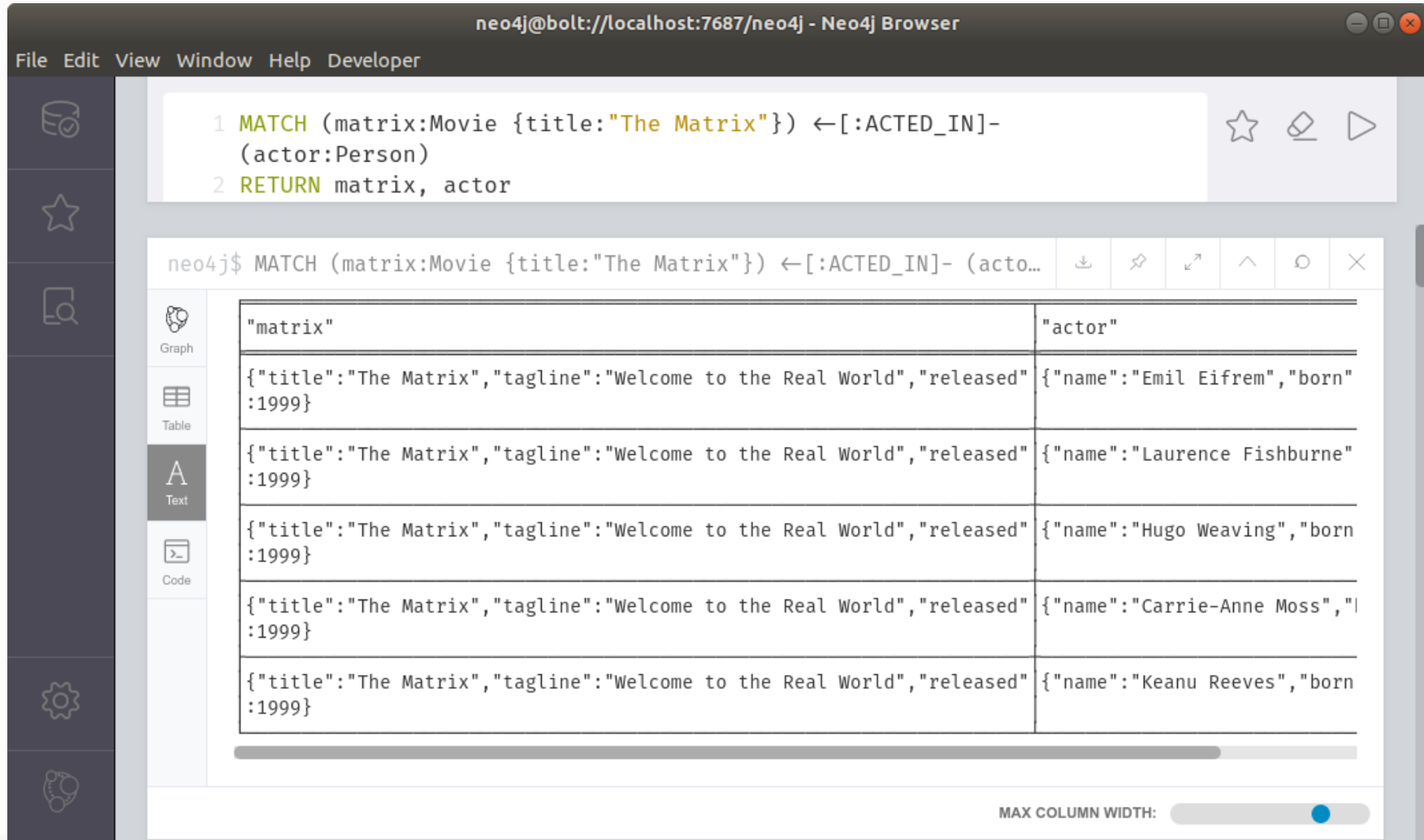
Neo4j Desktop

Visualização dos Resultados



Neo4j Desktop

Visualização dos Resultados



The screenshot displays the Neo4j Browser interface. At the top, the title bar reads "neo4j@bolt://localhost:7687/neo4j - Neo4j Browser". Below it is a menu bar with "File", "Edit", "View", "Window", "Help", and "Developer". A left sidebar contains icons for Graph, Table, Text, Code, and Settings. The main area shows a Cypher query in a text editor:

```
1 MATCH (matrix:Movie {title:"The Matrix"}) ←[:ACTED_IN]-  
  (actor:Person)  
2 RETURN matrix, actor
```

Below the query editor, the command line shows the executed query: `neo4j$ MATCH (matrix:Movie {title:"The Matrix"}) ←[:ACTED_IN]- (acto...`. The results are displayed in a table view with two columns: "matrix" and "actor".

"matrix"	"actor"
{"title":"The Matrix","tagline":"Welcome to the Real World","released":1999}	{"name":"Emil Eifrem","born":1999}
{"title":"The Matrix","tagline":"Welcome to the Real World","released":1999}	{"name":"Laurence Fishburne","born":1999}
{"title":"The Matrix","tagline":"Welcome to the Real World","released":1999}	{"name":"Hugo Weaving","born":1999}
{"title":"The Matrix","tagline":"Welcome to the Real World","released":1999}	{"name":"Carrie-Anne Moss","born":1999}
{"title":"The Matrix","tagline":"Welcome to the Real World","released":1999}	{"name":"Keanu Reeves","born":1999}

At the bottom right, there is a "MAX COLUMN WIDTH:" slider.

Cypher

Busca por Relacionamentos

- Outro exemplo: Devolva todos os personagens dos filmes no BD

```
MATCH (actor:Person) -[rel:ACTED_IN] -> (movie:Movie)
RETURN rel.roles
```

Obs.: os personagens aparecem na propriedade `roles` dos relacionamentos com rótulo `ACTED_IN`.

Cypher

Busca por Padrões

- Combinando buscas por nós com buscas por relacionamentos, é possível especificar buscas por padrões
- Exemplo de padrão simples: verifique se o Keanu Reeves fez o personagem Neo no filme The Matrix

```
MATCH (matrix:Movie {title:"The Matrix"} )  
      <-[role:ACTED_IN {roles:["Neo"]}]-  
      (keanu:Person {name:"Keanu Reeves"})  
  
RETURN matrix, role, keanu
```

Cypher

Busca por Padrões

- Também é possível filtrar nós e relacionamentos usando a cláusula WHERE
- Ela funciona da mesma forma como na SQL
- Exemplo: a consulta abaixo busca os filmes nos quais o Keanu Reeves atuou fazendo um personagem chamado Neo

```
MATCH (keanu:Person) -[r:ACTED_IN] -> (movie)
WHERE keanu.name="Keanu Reeves"
AND "Neo" IN r.roles
RETURN movie.title
```

Cypher

Variáveis em Buscas

- Para devolver todos os nós que possuem relacionamentos com outros nós:

```
MATCH (n) --> (m)
```

```
RETURN n, m;
```

- O resultado de uma busca pode ser armazenado em uma variável:

```
MATCH cast = (:Person) -[:ACTED_IN] -> (:Movie)
```

```
RETURN cast
```

Cypher

Criação de Nós

- Exemplo: criação de um novo nó de pessoa

```
CREATE (new:Person {name: "Kelly R B"})  
RETURN new
```

- Consulta para verificar se o nó criado está no grafo

```
MATCH (kelly:Person {name:"Kelly R B"})  
RETURN kelly.name
```

ou

```
MATCH (kelly:Person)  
WHERE kelly.name="Kelly R B"  
RETURN kelly.name
```

Cypher

Criação de Relacionamentos

- Primeiro, vamos criar um novo filme:

```
CREATE (movie:Movie {title: "Mystic River", released:1993})
```

- Agora, vamos incluir no BD o fato do Kevin Bacon ter atuado como Sean em Mystic River:

```
MATCH (kevin:Person) WHERE kevin.name = "Kevin Bacon"
```

```
MATCH (mystic:Movie) WHERE mystic.title = "Mystic River"
```

```
CREATE (kevin)-[r:ACTED_IN {roles:["Sean"]}]->(mystic)
```

```
RETURN mystic, r, kevin
```

Cypher

Criação de Relacionamentos

- Vamos incluir mais um novo relacionamento: uma avaliação para o filme `Mystic River`

```
MATCH (kelly:Person {name:"Kelly R B"}),  
      (movie:Movie {title:"Mystic River"})  
  
CREATE (kelly)-[r:REVIEWED {rating:80,  
  summary:"tragic character movie"}]->(movie)  
  
RETURN kelly, r, movie
```

Cypher

Criação de Relacionamentos

- O comando abaixo tem resultado igual ao do slide anterior:

```
MATCH (kelly:Person), (movie:Movie)
WHERE kelly.name="Kelly R B"
      AND movie.title="Mystic River"
CREATE (kelly)-[r:REVIEWED{rating:80,
      summary:"tragic character movie"}]-(movie)
RETURN kelly, r, movie
```

Cypher

Merge de Relacionamentos

- Se executamos um comando de criação de relacionamentos mais de uma vez, mais de uma aresta é criada entre o mesmo par de nós
- Para evitar esse problema, podemos usar o comando MERGE no lugar do CREATE
- O MERGE tem semântica *get-or-create*, ou seja, primeiro ele busca pela aresta. Se a encontrar no grafo, devolve-a como resposta. Senão, ele cria a aresta.
- Exemplo:

```
MATCH (kelly:Person {name:"Kelly R B"}),  
      (movie:Movie {title:"Mystic River"})  
MERGE (kelly)-[r:REVIEWED {rating:80,  
    summary:"tragic character movie"}]->(movie)  
  
RETURN kelly, r, movie
```

Cypher – Adição de Novas Propriedades

- O operador **SET** pode ser usado para incluir propriedades
- Exemplo: adiciona uma *tagline* ao filme `Mystic River`

```
MATCH (movie:Movie)
```

```
WHERE movie.title = "Mystic River"
```

```
SET movie.tagline = "We bury our sins here, Dave. We wash  
them clean."
```

```
RETURN movie.title AS title,  
       movie.tagline AS tagline
```

Cypher

Modificação de Propriedades

- O operador **set** também pode ser usado para mudar o valor de propriedades que já existem
- Exemplo: muda o nome do papel do Kevin Bacon em Mystic River para Sean Devine:

```
MATCH (kevin)-[r:ACTED_IN]->(mystic)
WHERE kevin.name = "Kevin Bacon"
      AND mystic.title = "Mystic River"
SET r.roles = ["Sean Devine"]
RETURN mystic, r, kevin
```

Cypher

Remoção de Nós e Relacionamentos

- O comando **DELETE** remove nós ou relacionamentos . Exemplo 1:

```
MATCH (kelly:Person {name:"Kelly R B"})
```

```
DELETE kelly
```

- Exemplo 2: o comando a seguir remove do BD o nó da `Kelly R B` e todos os relacionamentos desse nó (caso ele tenha algum)

```
MATCH (kelly:Person {name:"Kelly R B"})
```

```
OPTIONAL MATCH (kelly)-[r]-()
```

```
DELETE kelly,r
```

Cypher

Remoção de Nós e Relacionamentos

- Como a operação de remover um nó juntamente com seus relacionamentos é bastante comum, foi incluída no Cypher uma variação do `DELETE`, o `DETACH DELETE`, que faz a remoção completa:

```
MATCH (kelly:Person {name:"Kelly R B"})
```

```
DETACH DELETE kelly
```

Cypher - OPTIONAL MATCH

- O comando abaixo removerá o nó da Kelly somente se ele tiver ao menos um relacionamento:

```
MATCH (kelly:Person {name:"Kelly R B"})
```

```
MATCH (kelly)-[r]-()
```

```
DELETE kelly,r
```

- Já o comando abaixo removerá o nó da Kelly mesmo se ele não tiver relacionamentos:

```
MATCH (kelly:Person {name:"Kelly R B"})
```

```
OPTIONAL MATCH (kelly)-[r]-()
```

```
DELETE kelly,r
```

O **OPTIONAL MATCH** , assim como o **MATCH**, busca por um padrão. A diferença é que, quando o padrão não é encontrado, ele devolve NULL nas variáveis envolvidas (comportamento similar ao da junção externa da SQL).

Cypher

Múltiplos MATCHes

- As três formas de especificar o padrão com a cláusula MATCH mostradas abaixo geram resultados equivalentes:

```
MATCH (kelly:Person {name:"Kelly R B"})
```

```
MATCH (kelly)-[r]-()
```

```
DELETE kelly,r
```

```
MATCH (kelly:Person {name:"Kelly R B"}), (kelly)-[r]-()
```

```
DELETE kelly,r
```

```
MATCH (kelly:Person {name:"Kelly R B"})-[r]-()
```

```
DELETE kelly,r
```

Cypher

Ordenação dos Resultados

- A cláusula ORDER BY ordena os resultados de uma consulta
- Funcionamento semelhante à da SQL
- Exemplo: a consulta abaixo mostra os anos de nascimento de todas as pessoas em ordem decrescente

```
MATCH (person:Person)
RETURN person.name, person.born
ORDER BY person.born DESC
```

Cypher

Limitando o Tamanho da Resposta

- A consulta abaixo devolve a “terceira página” (cada página com 5 atores) dos dados de atores e suas atuações:

```
MATCH (actor:Person) -[:ACTED_IN]->(movie:Movie)
RETURN actor.name AS Actor,
        movie.title AS Movie

SKIP 10 LIMIT 5
```

Cypher

Eliminando Repetições

- Usa-se o operador DISTINCT para eliminar repetições na resposta de uma consulta.
- Exemplo: Mostra os 5 atores mais velhos.

```
MATCH (actor:Person) -[:ACTED_IN]->()  
  
RETURN DISTINCT actor  
  
ORDER BY actor.born  
  
LIMIT 5
```

Cypher

Buscas mais avançadas

- Encontra os nomes dos atores que atuaram em filmes em que o Tom Hanks atuou e que são mais velhos que o Tom Hanks:

```
MATCH (tom:Person)
      -[:ACTED_IN]->()-[:ACTED_IN]-
      (actor:Person)
WHERE tom.name="Tom Hanks"
AND actor.born < tom.born
RETURN DISTINCT actor.name AS Name
```

Cypher – Buscas mais avançadas

- Encontra os nomes e a diferença de idade dos atores que atuaram em filmes em que o Tom Hanks atuou e que são mais velhos que o Tom Hanks:

```
MATCH (tom:Person {name:"Tom Hanks"})  
      -[:ACTED_IN]->(movie:Movie),  
      (movie)<-[:ACTED_IN]-(actor:Person)  
WHERE actor.born < tom.born  
RETURN DISTINCT actor.name AS Name,  
      (tom.born - actor.born) AS diff
```

Cypher – Buscas mais avançadas

- Encontra os nomes dos atores que são também diretores e que atuaram em filmes em que o Gene Hackman atuou:

```
MATCH (gene:Person)
      -[:ACTED_IN]->()<-[:ACTED_IN]-
      (other:Person)
WHERE gene.name="Gene Hackman"
AND EXISTS ( (other)-[:DIRECTED]->() )
RETURN DISTINCT other
```

Cypher – Busca por Padrões com Caminhos de Tamanho Variável

- Devolve todos os pares de nós que estão conectados, mesmo que seja por meio de relacionamentos indiretos de qualquer distância:

```
MATCH (node1) -[*] - (node2)  
RETURN node1, node2
```

Obs.: A execução dessa consulta sobre o BD usado para os exemplos da aula é bastante custosa computacionalmente (mesmo o BD sendo pequeno). Cuidado ao testá-la, pois ela pode consumir toda a memória RAM do seu computador.

Cypher – Busca por Padrões com Caminhos de Tamanho Variável

Variações

- Encontra pares de nós que estão relacionados por meio de um caminho de tamanho **4**:

```
MATCH (a) -[*4] -> (b)
```

- Encontra pares de nós que estão relacionados por meio de um caminho de tamanho **entre 1 e 4**:

```
MATCH (a) -[*1..4] -> (b)
```

- Encontra pares de nós que estão relacionados (mesmo que indiretamente) por meio de relacionamentos do tipo `LIKE`

```
MATCH (a) -[:LIKE *] -> (b)
```

- Encontra pares de nós que estão relacionados por meio de relacionamentos do tipo `LIKE` em segundo grau **ou** relacionamentos diretos do tipo `KNOWS` em primeiro grau

```
MATCH (a) -[:LIKE *2 | :KNOWS] -> (b)
```

Cypher

Extraindo Dados de Caminhos

- A função **nodes** devolve os nós existentes em um caminho (subgrafo)
- Exibe somente os nós dos caminhos devolvidos na consulta (MATCH):

```
MATCH p = (actor:Person)
        -[:ACTED_IN]->(movie:Movie)<-[:DIRECTED]-
        (director:Person)

RETURN nodes (p) ;
```

- Além da função **nodes**, temos também:
 - **length (p)** – devolve o tamanho do caminho p
 - **rels (p)** – devolve apenas os relacionamentos no caminho p

Cypher

Agrupamento & Agregação

- Funções de agregação:
 - **count(x)**: conta o número de ocorrências
 - **min(x)**: obtém o menor valor
 - **max(x)**: obtém o maior valor
 - **avg(x)**: obtém a média de valores numéricos
 - **sum(x)**: soma valores
 - **collect(x)**: coloca todos os valores em um vetor
- O agrupamento é feito de forma automática (não é declarado na consulta)

Cypher

Agrupamento & Agregação

- Mostra todos os títulos dos filmes dos quais um ator participou

```
MATCH (person:Person) -[:ACTED_IN]->(movie:Movie)
RETURN person.name, collect(movie.title)
```

- Mostra o nome de todos os diretores com os quais um ator trabalhou

```
MATCH (person:Person) -[:ACTED_IN]->
      (movie:Movie) <-[:DIRECTED]-
      (director:Person)
RETURN person.name,
       collect(DISTINCT director.name) as directors
```

Cypher

Agrupamento & Agregação

- Devolve o número de filmes em que um ator trabalhou junto com um mesmo diretor:

```
MATCH (actor:Person)
      -[:ACTED_IN]->(movie:Movie)<-[:DIRECTED]-(director:Person)
RETURN actor.name, director.name, count(movie);
```

- Devolve os 10 atores que mais atuaram em filmes

```
MATCH (person:Person)-[:ACTED_IN]->(movie:Movie)
RETURN person.name, count(movie)
ORDER BY count(movie) DESC
LIMIT 10;
```

Cypher

Cláusula WITH

- A cláusula WITH permite que as partes da consulta sejam encadeadas, canalizando os resultados de uma para serem usados como entrada ou critérios na próxima
 - As variáveis não incluídas na cláusula WITH não são transferidas para o restante da consulta.
 - O caracter curinga * pode ser usado para incluir todas as variáveis que estão atualmente no escopo.
- Exemplo 1:

```
MATCH (person:Person)-[:ACTED_IN]->(:Movie{title:"The Matrix"})
WITH person, toUpper(person.name) AS upperCaseName
WHERE upperCaseName STARTS WITH 'K'
RETURN person.name
```

<https://neo4j.com/docs/cypher-manual/current/clauses/with/>

Cypher

Cláusula WITH

- A cláusula WITH permite que as partes da consulta sejam encadeadas, canalizando os resultados de uma para serem usados como entrada ou critérios na próxima
 - As variáveis não incluídas na cláusula WITH não são transferidas para o restante da consulta.
 - O caracter curinga * pode ser usado para incluir todas as variáveis que estão atualmente no escopo.
- Exemplo 2: Devolve o título dos filmes que tiveram mais que 10 atores

```
MATCH (person:Person) -[:ACTED_IN]-> (movie:Movie)
```

```
WITH movie, count(person) AS numberOfActors
```

```
WHERE numberOfActors > 10
```

```
RETURN movie.title
```

<https://neo4j.com/docs/cypher-manual/current/clauses/with/>

Cypher

Múltiplos WITHs e MATCHes

- Uma consulta pode ter múltiplas cláusula WITH e MATCHes intercaladas
- Exemplo - Para os atores que também são diretores e que foram dirigidos por pelo menos 2 diretores, mostre os nomes dos atores que eles dirigiram:

```
MATCH (person:Person)-[:ACTED_IN]->(:Movie)<-[:DIRECTED]-(director:Person)
```

```
WITH person, count(distinct director) AS count_directors
```

```
WHERE count_directors >= 2
```

```
MATCH (person)-[:DIRECTED]->(:Movie)<-[:ACTED_IN]-(actor:Person)
```

```
WITH person, collect(distinct actor) AS actors, count_directors
```

```
RETURN person.name, count_directors, actors
```

<https://neo4j.com/docs/cypher-manual/current/clauses/with/>

Para Finalizar a Aula:

Distribuição de Dados em Neo4j

- No Neo4j, é possível distribuir um BD de grafo em um *cluster* tanto por meio de replicação quanto por particionamento
- Assim como ocorre em outros NoSQL, a replicação no Neo4j tem como objetivos:
 - Aumentar a segurança (por meio da redundância de dados)
 - Escalar as operações de leitura: <https://neo4j.com/docs/operations-manual/current/clustering/>
- O particionamento (*sharding*) é feito por meio do mecanismo de *Composite Databases*
 - Um *Composite Database* do Neo4j é um agrupamento lógico que fornece um ponto de acesso unificado a múltiplas bases de dados subjacentes (conhecidas como “constituintes”), que podem estar no mesmo SGBD do Neo4j ou em SGBDs remotos e distintos.
 - Ele é um recurso da edição Enterprise do Neo4j, projetado para oferecer escalabilidade por meio de federação de dados e *sharding*.
 - <https://neo4j.com/docs/operations-manual/current/scalability/composite-databases/concepts/>

Neo4j

Replicação de Dados

- Provê escalabilidade horizontal para operações de leitura por meio de replicação primário-secundário
- Todas as escritas num *cluster* são coordenadas pelo nó primário, mas o Neo4j permite escritas em nós secundários
 - Mas o nó secundário precisa propagar as atualizações para o primário antes que se possa responder ao cliente confirmando a operação
 - Por causa do tráfego de rede adicional e o protocolo de coordenação, o tempo da escrita em nós secundários pode ser uma ordem de magnitude maior que o da escrita direta no nó primário
- Nas versões mais novas do Neo4j, é possível definir quórum de escrita, para aumentar a garantia de durabilidade das escritas feitas no nó primário

Neo4j

Replicação de Dados

- Em BDs com replicação, depois de uma operação de escrita, existe uma janela de inconsistência
 - Tempo de propagação da operação para todas as réplicas
- Durante a janela de inconsistência, os clientes poderão acessar réplicas com dados desatualizados
- Em algumas aplicações, é necessário garantir consistência do tipo leia-suas-próprias-escritas
 - A garantia de que um cliente que faz uma consulta a dados logo depois de fazer uma operação de escrita “enxergará” os dados que ele acabou de escrever
- No Neo4j, para garantir esse tipo de consistência, uma operação de escrita de um cliente pode ser roteada para o mesmo nó secundário que será usado para atender a consulta subsequente do cliente.

Referências Sobre o Neo4J

- Documentação do Neo4J
 - <https://neo4j.com/docs/>
- “Treinamentos” oficiais:
 - <https://neo4j.com/graphacademy/>
- Livros (disponíveis para download gratuito!) :
 - **“Graph Databases - The Definitive Book on Graph Databases”, 2ª edição (2015)**
<http://graphdatabases.com/>
 - **“The Definitive Guide to Graph Databases – for the RDBMS developers”**
<https://neo4j.com/resources/rdbms-developer-graph-white-paper/>

Obs.: compara BDs de grafos com BDs relacionais



Referências sobre a Cypher

- Documentação
 - <https://neo4j.com/docs/cypher-manual/current/introduction/>
- Cartão de referência
 - <https://neo4j.com/docs/cypher-cheat-sheet/5/all/>

Obrigada!

Profª Drª Kelly Rosa Braghetto | [linkedin.com/in/kelly-rosa-braghetto/](https://www.linkedin.com/in/kelly-rosa-braghetto/)

